# WolfISMS

Wolf's Information Security Management System

# Docker Installation

## Version 3.92

As of: October 2026

© 2012–2026 Wolfgang Schuster

# 1. Overview

This guide describes how to run WolfISMS as a Docker container behind the Nginx Proxy Manager (NPM). NPM handles TLS termination and certificate management, while WolfISMS itself speaks plain HTTP inside the Docker network.

It is aimed at administrators who already operate a Docker environment. For the classic installation directly on an Ubuntu or Debian server with Apache and mod_wsgi, see chapter 3 of the administration documentation instead.

## 1.1 Target Architecture

| Component | Role |
|---|---|
| **Nginx Proxy Manager** | Accepts HTTP (80) and HTTPS (443) from the outside, terminates TLS, manages the Let's Encrypt certificates and forwards requests to the WolfISMS container on the internal network. The administration interface runs on port 81 |
| **WolfISMS container** | The application itself. It is not directly reachable from outside, only through the Docker network proxy |
| **Database** | MariaDB or MySQL – either as its own container or as an existing database server, switched via DB_MODUS (see chapter 5) |
| **Docker network "proxy"** | Connects NPM and WolfISMS. It is created manually once and referenced as an external network by both Compose projects |

## 1.2 Why HTTPS Is Mandatory

WolfISMS sets its session cookies with the attribute Secure=True. Browsers transmit such cookies only over encrypted connections. Over plain HTTP the login page will therefore load, but signing in fails because the session cannot be retained. A valid certificate is not a recommendation but an operating prerequisite.

**Reverse proxy support is built in:**
The application evaluates the headers X-Forwarded-Proto and X-Forwarded-Host (ProxyFix in isms.py). It therefore correctly recognises that it runs behind a TLS termination point and generates absolute links using https and the public host name. The Nginx Proxy Manager sets these headers by default – no additional configuration is required.

© 2012–2026 Wolfgang Schuster

# 2. Prerequisites

| Component | Requirement |
|---|---|
| **Docker Engine** | Current version with the Compose V2 plug-in (docker compose, not docker-compose) |
| **Operating system** | Any Docker host (Linux recommended) |
| **Public DNS record** | The desired host name, e.g. isms.your-domain.com, must resolve to the public IP address of the Docker host |
| **Open ports** | Ports 80 and 443 must be reachable from the outside. Port 80 is required for the Let's Encrypt ACME challenge |
| **Port 81** | The NPM administration interface. It should not be publicly reachable but restricted to the internal network or VPN |
| **Installation package** | The WolfISMS repository or Docker package containing docker-compose.yml and .envdefault (provided by the vendor) |
| **Database** | MariaDB 10.11+ or MySQL 8.0+ – as a container or as an external server |

**Do not expose port 81 to the internet:**
The administration interface of the Nginx Proxy Manager controls all certificates and forwarding rules. Restrict access to the internal network using the host firewall, and change the default credentials immediately after the first sign-in.

© 2012–2026 Wolfgang Schuster

# 3. Step 1 – Create the Docker Network

Both Compose projects – the Nginx Proxy Manager and WolfISMS – reference the same external network. It is created once on the Docker host:

```
docker network create proxy
```

Verify that it exists:

```
docker network ls | grep proxy
```

> The network must exist before the first docker compose up. Without it, both projects abort with the message "network proxy declared as external, but could not be found".

© 2012–2026 Wolfgang Schuster

# 4. Step 2 – Deploy the Nginx Proxy Manager

Create a dedicated directory for the reverse proxy on the Docker host and place the file docker-compose.yml inside it:

```
mkdir -p /opt/npm
cd /opt/npm
```

Contents of /opt/npm/docker-compose.yml:

```
services:
  npm:
    image: jc21/nginx-proxy-manager:2
    restart: unless-stopped
    ports:
      - "80:80"     # HTTP + ACME challenge
      - "443:443"   # HTTPS
      - "81:81"     # administration interface
    volumes:
      - npm_data:/data
      - npm_letsencrypt:/etc/letsencrypt
    networks:
      - proxy

networks:
  proxy:
    external: true
    name: proxy

volumes:
  npm_data:
  npm_letsencrypt:
```

Then start the container:

```
docker compose up -d
```

## 4.1 First Sign-In

The administration interface is then available at http://<docker-host>:81. On first start the Nginx Proxy Manager creates a default account whose credentials are documented with the image.

> Change the email address and password of the default account immediately after the first sign-in. As long as the default values apply, anyone with network access to port 81 can modify the certificates and forwarding rules of the entire environment.

© 2012–2026 Wolfgang Schuster

# 5. Step 3 – Deploy WolfISMS

The WolfISMS package contains docker-compose.yml as well as the template .envdefault. The actual configuration is done exclusively through environment variables in the file .env.

## 5.1 Obtain the Package and Create the Configuration

```
git clone https://github.com/wolfschus/wolfisms.git
cd WolfISMS
cp .envdefault .env
```

Then open .env and adapt the values to your environment. Chapter 8 contains the complete reference of all variables.

If you need your own certificates – for example to connect to an external database server – copy them into the directory db-certs/ (see Database: Internal or External). This is not required for outward-facing TLS termination; the Nginx Proxy Manager takes care of that.

## 5.2 Database: Internal or External

The variable DB_MODUS in the .env decides whether MariaDB runs as a container within the stack. docker-compose.yml includes the file docker-compose.db-<DB_MODUS>.yml for this. If the entry is missing – for example in a .env from an older version –, internal applies; existing installations therefore keep running unchanged.

| DB_MODUS | Behaviour |
|---|---|
| **intern (default)** | docker-compose.db-intern.yml: container db with MariaDB 11.4 and volume db_data. On first start it creates database and user and imports a dump from ./db-init. DB_HOST stays empty, DB_ROOT_PASSWORD is mandatory |
| **extern** | docker-compose.db-extern.yml: no database container is started. App and cron connect to DB_HOST and DB_PORT; DB_ROOT_PASSWORD is not used |

For an external database, database and user must already exist on the server, for example:

```
CREATE DATABASE isms CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'isms'@'%' IDENTIFIED BY '<password>';
GRANT ALL PRIVILEGES ON isms.* TO 'isms'@'%';
```

- **Connection**: DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD in the .env; DB_TYPE stays mariadb (or mysql). The values apply to app and cron alike
- **TLS**: DB_SSL_MODE=required encrypts without checking the certificate, verify-full checks certificate and host name against DB_SSL_CA. The certificate files belong in the folder db-certs/ next to docker-compose.yml; inside the container they are found under /var/www/isms/db-certs (read-only), e.g. DB_SSL_CA=/var/www/isms/db-certs/db-ca.pem
- **Data backup**: the backup in the admin area uses the same connection. The user also needs read access to routines, triggers and events for it – GRANT ALL on the database is sufficient
- **Moving data**: an existing installation moves by importing its dump (backup in the admin area) on the new server before DB_MODUS is switched
- **Prerequisite**: include and the optional dependency (depends_on … required: false) require Docker Compose 2.20 or newer

© 2012–2026 Wolfgang Schuster

**Switching the mode:**
First docker compose down (the volume db_data is kept), then change DB_MODUS in the .env, then docker compose up -d. The file docker-compose.db-extern.yml contains the service db only as a disabled placeholder (profile nie-starten) – please do not add that profile to COMPOSE_PROFILES.

## 5.3 Start the Containers

```
docker compose up -d
```

Check the start-up:

```
docker compose ps
docker compose logs -f
```

## 5.4 Connecting to the Reverse Proxy

For the Nginx Proxy Manager to reach the application container, both must be on the proxy network and the container must be addressable under a stable name. That name is set through the variable PROXY_ALIAS and entered in the proxy host as the forward hostname. The supplied docker-compose.yml already joins the network; the following example shows the relevant sections:

```
services:
  wolfisms:
    # ... image, volumes, depends_on ...
    env_file: .env
    restart: unless-stopped
    networks:
      proxy:
        aliases:
          - ${PROXY_ALIAS}    # e.g. wolfisms-app

networks:
  proxy:
    external: true
    name: proxy
```

**No port mapping for the application:**
The WolfISMS container needs no ports entry. It is addressed exclusively through the internal proxy network. An additionally published port would make the application reachable directly over HTTP, bypassing TLS termination – and, because of the Secure cookies, unusable for signing in as well.

The authoritative docker-compose.yml is part of the installation package. The example above shows only the sections relevant to the proxy connection and does not replace the supplied file.

© 2012–2026 Wolfgang Schuster

# 6. Step 4 – Configure the Proxy Host and Certificate

Publication is then configured in the NPM interface under Hosts → Proxy Hosts → Add Proxy Host.

## 6.1 "Details" Tab

| Field | Value |
|---|---|
| **Domain Names** | isms.your-domain.com |
| **Scheme** | http |
| **Forward Hostname** | wolfisms-app (the name set in PROXY_ALIAS) |
| **Forward Port** | 8000 |
| **Block Common Exploits** | enabled |
| **Websockets Support** | enabled |

## 6.2 "SSL" Tab

- **SSL Certificate**: select "Request a new SSL Certificate"
- **Force SSL**: enable – redirects HTTP requests to HTTPS automatically
- **HTTP/2 Support**: enable
- **Email address**: provide it for the Let's Encrypt expiry notifications and accept the terms of service

For the standard challenge Let's Encrypt requires the domain name to resolve publicly to the host IP address and port 80 to be reachable from the outside. If that is not the case – for example in a purely internal environment – configure a DNS challenge with your DNS provider instead, or upload your own certificate.

[ IMAGE PLACEHOLDER ]

**Figure: Creating a proxy host**

*Screenshot of the "Add Proxy Host" dialog in the Nginx Proxy Manager with the "Details" tab filled in*

[ IMAGE PLACEHOLDER ]

**Figure: Requesting a certificate**

*Screenshot of the "SSL" tab with the options "Request a new SSL Certificate", "Force SSL" and "HTTP/2 Support"*

© 2012–2026 Wolfgang Schuster

## 6.3 Verify the Installation

| Check | Expected result |
|---|---|
| **Open https://isms.your-domain.com** | The login page appears and the certificate is shown as valid |
| **Request the same address over HTTP** | Automatic redirect to HTTPS (Force SSL) |
| **Sign in** | Redirect to the dashboard – if signing in fails, see chapter 9 |
| **Container log** | docker compose logs without traceback entries |
| **Admin area** | The "Updates" menu entry is absent – this is intended in container mode (see chapter 7) |

**Immediately after the first sign-in:**
1) Change the default password of the admin user
2) Install the licence file customer.lic (Admin → Licence)
3) Restart the container so the licence and the secret key derived from it take effect: docker compose restart

© 2012–2026 Wolfgang Schuster

# 7. Operating in Container Mode

With the environment variable WOLFISMS_CONTAINER=1 the application recognises that it runs inside a container (config.container_mode). This changes its behaviour in two places.

## 7.1 In-App Updater Disabled

The entire update area including /admin/updates is disabled and responds with HTTP 404; the tile in the admin area is hidden. This is intentional: an update applied inside a running container would be lost the next time the image is restarted.

In Docker operation, updates are applied exclusively through a new image:

```
docker compose pull
docker compose up -d
```

## 7.2 Configuration Comes Only From the Environment

Outside Docker, config.py reads the credentials from the file /etc/wolfisms/secret.env. Inside the container that file usually does not exist – which is not an error: if it is missing, the application uses only the environment variables that are set. Those are exactly what .env provides through docker compose.

**Different path:**
If you do want to use an env file inside the container, its path can be set with the variable WOLFISMS_ENV_FILE. Environment variables that are already set always take precedence over the values from the file.

## 7.3 Data Persistence and Backup

The database and the upload directory must be persisted. Both have to be mounted as volumes, otherwise they are lost when the container is replaced.

| Data | Location | Note |
|---|---|---|
| **Database** | Volume of the database container or external database server | Contains all ISMS content |
| **Uploaded files** | UPLOAD_DIR (default /var/www/isms/wolfisms/uploads) | Attachments, playbooks, logo, templates and letterhead |
| **Sessions** | SESSION_DIR (default /var/www/isms/wolfisms/sessions) | Must be writable; losing it merely signs all users out. The application sets the directory to 0700 and the session files to 0600 at startup |
| **Licence file** | license/customer.lic in the application directory | After switching images, check that it is still present |

The backup itself works as in the classic installation, through the admin area at /admin/backup or the direct link /backup/download. The archive contains the database dump and the upload

© 2012–2026 Wolfgang Schuster

directory; application code, configuration and the licence file are not included and must be backed up separately. Chapter 5 of the administration documentation covers the details.

## 7.4 Logs

| Purpose | Command |
|---|---|
| **Follow live output** | docker compose logs -f |
| **Application only** | docker compose logs -f wolfisms |
| **Last 200 lines** | docker compose logs --tail=200 |
| **Proxy access and certificates** | docker compose logs -f npm (in the directory /opt/npm) |

© 2012–2026 Wolfgang Schuster

# 8. Configuration Reference (.env)

All parameters are set as environment variables. They are resolved in this order: environment variables that are already set, then the env file, and finally the defaults defined in the code. There is deliberately no default for DB_PASSWORD – if the variable is missing, the application will not start.

## 8.1 Mandatory Settings

| Variable | Example | Meaning |
|---|---|---|
| **DB_PASSWORD** | (strong password) | Database password – the only value without a default |
| **DB_USER** | isms | Database user (default: wolf) |
| **DB_HOST** | db | Empty = internal container db; with DB_MODUS=extern host name or IP of the database server |
| **DB_NAME** | isms | Name of the database |
| **ISMS_BASE_URL** | https://isms.your-domain.com | Public base URL – required for links in reminder emails |
| **FLASK_SECRET_KEY** | (64 hex characters) | Fallback session key. If a valid licence is present it is ignored and the key is derived from the licence ID |
| **WOLFISMS_CONTAINER** | 1 | Enables container mode (see chapter 7) |
| **PROXY_ALIAS** | wolfisms-app | Network alias of the container; enter it as the forward hostname in the proxy host |

Generate a secure random value for FLASK_SECRET_KEY:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

## 8.2 Database

| Variable | Default | Meaning |
|---|---|---|
| **DB_MODUS** | intern | intern = MariaDB container within the stack, extern = existing database server (see the section Database: Internal or External) |
| **DB_TYPE** | mariadb | mariadb or mysql; mssql is reserved and currently rejected at start-up |
| **DB_PORT** | (empty) | Empty = 3306 |
| **DB_ROOT_PASSWORD** | – | intern only: root password of the database container |
| **DB_SSL_MODE** | (empty) | empty = unchanged, disabled, required or verify-full |
| **DB_SSL_CA** | (empty) | CA file inside the container, mandatory for verify-full |
| **DB_SSL_CERT / DB_SSL_KEY** | (empty) | Client certificate and key, if the server requires them |

© 2012–2026 Wolfgang Schuster

Invalid values – an unknown DB_TYPE, a wrong DB_SSL_MODE or verify-full without DB_SSL_CA – stop the application from starting at all, with a clear message in the log.

## 8.3 Paths and Instance

| Variable | Default | Meaning |
|---|---|---|
| **WEBPATH** | (empty) | URL path prefix. Leave empty when running under a dedicated domain; set it only if the application runs under a sub-path such as /isms |
| **PATHNAME** | isms | Directory name of the instance |
| **UPLOAD_DIR** | /var/www/isms/wolfisms/uploads | Path for uploaded files – mount as a volume |
| **SESSION_DIR** | /var/www/isms/wolfisms/sessions | Path for server-side sessions – must be writable |
| **WOLFISMS_ENV_FILE** | /etc/wolfisms/secret.env | Alternative path of an env file inside the container |

## 8.4 Language

| Variable | Default | Meaning |
|---|---|---|
| **DEUTSCH** | True | Allow the German user interface |
| **ENGLISCH** | False | Allow the English user interface. With DEUTSCH=0 and ENGLISCH=1 the installation runs in English only |

## 8.5 Single Sign-On (optional)

| Variable | Meaning |
|---|---|
| **MS_AUTH** | 1 = enable sign-in through Microsoft Entra ID |
| **GOOGLE_AUTH** | 1 = enable sign-in through Google |
| **SSO_ALLOWED_DOMAINS** | Comma-separated allow list of permitted email domains. An empty list admits nobody – that is the safe default |
| **SSO_AUTO_CREATE** | 1 = create unknown SSO users automatically (default 0) |
| **MS_TENANT_ID / MS_CLIENT_ID / MS_CLIENT_SECRET / MS_REDIRECT_URI** | Credentials and callback URL for Microsoft |
| **GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI** | Credentials and callback URL for Google |

The callback URL must point to the public host name served by the reverse proxy – not to the container name.

© 2012–2026 Wolfgang Schuster

## 8.6 Further Variables

| Variable | Meaning |
|---|---|
| **UPDATE_PROXY** | Proxy for outbound connections. In container mode it has no effect on the disabled updater |
| **RATELIMIT_STORAGE_URI** | Storage for the rate limit counters. The default memory:// counts per worker process – with several workers every limit is multiplied accordingly. For such installations point it at shared storage, e.g. redis://redis:6379 |
| **ISMS_DEBUG** | 1 = switch on Flask debug mode. For troubleshooting only; remove it afterwards |

ISMS_DEBUG must not remain set in production. Debug mode enables PROPAGATE_EXCEPTIONS, disables template caching and is a finding in every hardening baseline.

© 2012–2026 Wolfgang Schuster

# 9. Troubleshooting

| Symptom | Cause and remedy |
|---|---|
| **"network proxy declared as external, but could not be found"** | The Docker network is missing. Create it with docker network create proxy (chapter 3) and start the containers again |
| **NPM reports "502 Bad Gateway"** | The application container cannot be reached under the configured forward hostname. Check that the container is running, that it is on the proxy network, and that PROXY_ALIAS matches the name entered in the proxy host |
| **The login page appears but signing in fails silently** | You are accessing the site over HTTP instead of HTTPS. Session cookies are Secure=True and are not set over unencrypted connections. Enable "Force SSL" in the proxy host |
| **The container does not start and the log mentions the database** | DB_PASSWORD is missing or the database is not ready yet. The variable is mandatory and deliberately has no default; check the start order via depends_on |
| “depends on undefined service db” or an error at include | Docker Compose is older than 2.20, or one of the files docker-compose.db-intern.yml / docker-compose.db-extern.yml is missing, or DB_MODUS has a value other than intern or extern |
| Log: “WARNUNG: Datenbank-Host 'db' ist nicht auflösbar” | DB_MODUS=extern is set but DB_HOST is empty – or DB_HOST=db with an external database. Set DB_HOST to the database server, or DB_MODUS=intern for the internal database |
| **The certificate request fails** | The DNS record does not point to the host, or port 80 is not reachable from the outside. Check both or switch to a DNS challenge |
| **Links inside the ISMS point to http:// or the container name** | The proxy does not pass X-Forwarded-Proto or X-Forwarded-Host, or ISMS_BASE_URL is set incorrectly. Check both |
| **The "Updates" menu entry is missing in the admin area** | Not an error: in container mode the in-app updater is disabled. Update through a new image (chapter 7.1) |
| **Demo mode persists after installing the licence** | The licence check is cached. Restart the container: docker compose restart |
| **Uploaded files disappeared after an update** | UPLOAD_DIR was not mounted as a volume. Add the volume and restore the files from the most recent backup |

© 2012–2026 Wolfgang Schuster

# 10. Checklist

| ✓ | Task |
|---|---|
| ☐ | DNS record resolves to the public IP address of the Docker host |
| ☐ | Ports 80 and 443 are reachable from the outside |
| ☐ | Port 81 is restricted to the internal network |
| ☐ | Docker network proxy created |
| ☐ | Nginx Proxy Manager started, default credentials changed |
| ☐ | .env created from .envdefault and fully populated |
| ☐ | DB_PASSWORD set and FLASK_SECRET_KEY replaced with a random value |
| ☐ | WOLFISMS_CONTAINER=1 set |
| ☐ | PROXY_ALIAS set and entered as the forward hostname in the proxy host |
| ☐ | ISMS_BASE_URL set to the public host name |
| ☐ | Volumes mounted for the database and UPLOAD_DIR |
| ☐ | With an external database: DB_MODUS=extern, DB_HOST/DB_PORT set, database and user created, log without connection errors |
| ☐ | Proxy host created, certificate requested, Force SSL and HTTP/2 enabled |
| ☐ | Sign-in over HTTPS tested successfully |
| ☐ | Default password of the admin user changed |
| ☐ | Licence file installed and the container restarted |
| ☐ | Second admin account created as emergency access |
| ☐ | Backup via /admin/backup tested and stored off-site |
| ☐ | ISMS_DEBUG not set |
| ☐ | .env not committed to a Git repository |

© 2012–2026 Wolfgang Schuster

# 11. Further Documentation

| Document | Content |
|---|---|
| **WolfISMS Administration Documentation** | Complete system administration: user and permission management, backup, licence management, REST API, two-factor authentication, single sign-on, AI settings and security notes. It also explains what differs in container mode compared with the classic installation on Apache with mod_wsgi |
| **WolfISMS User Documentation** | Description of all functional modules for end users |
| **WolfISMS REST API** | Interface description for connecting external systems: incidents, measures, assets, risks and suppliers – with all fields, filters, value lists and a complete example sync client |

**WolfISMS – Wolf's Information Security Management System**
© 2012–2026 Wolfgang Schuster
Update server: https://update.wolfisms.de
Version: 3.92 | As of: October 2026